PORTFOLIO · 26.09 기준

PORTFOLIO

```
const engineer = "최지현"; // Trusted AI Agent
```

★ Agent System Design ★ End-to-End 서비스 ★ Trusted AI

최지현 | AI가 들어갈 시스템 전체를 설계합니다

AI Agent Engineer · Backend

GitHub github.com/ttogle918 Email ttogle918@gmail.com Portfolio ttogle918.github.io

★ ABOUT ME ★

저는 **AI가 들어갈 시스템 전체**를 설계하는 AI 에이전트 엔지니어입니다.
ML 모델링과 NLP 파인튜닝부터 에이전트까지 폭넓게 다뤄 왔습니다.

01

★ 구조 설계

Agent System Design

에이전트 · 백엔드 · DB로 계층을 나누고, 계층마다 맡을 책임을 정해 설계합니다.

02

★ 신뢰성

Trusted AI (RAG & Validation)

근거 원문, 교차 검증, 벤치마크 측정으로 AI의 답을 믿을 수 있는지 확인합니다.

03

★ 도메인 적응력

End-to-End & Adaptability

보안 · 통관 · 채용 · 제조 · 보험 · 금융.
그 일을 하는 사람이 무엇을 확인하는지부터 찾습니다.

CONTENTS

About Me 02
01-1 FinAIQ - 은행 06
04 K-Bridge - 무역 규제 분석 (팀) 15
Future Plans 21
Skill 03
01-2 InsuQ - 보험 08
05 KOCRUIT - 채용 관리 (팀) 17
Contact 22
Career & Education 04
02 MaintQ - 설비 진단 · 부품 발주 10
06 KeyLens · 07 SpendQ · 08 보리 19
01 QMesh - 회사 간 에이전트 연결 05
03 SecureAI Engine - 코드 보안 13
하네스 · 09-10 비상교육 20

★ SKILL ★

프로젝트에서 직접 써 본 기술입니다. 핵심 스택은 **파란색**으로 표시했습니다.

★ AI 에이전트 · 오케스트레이션

LangGraph · RAG · MCP / FastMCP · A2A Protocol · Claude API · LangChain · Claude Code 멀티 에이전트 · Hybrid Search · Reranker · HyDE · pgvector / Qdrant / FAISS · GPT-4o · Gemini

★ Backend / Infra

FastAPI (Async) · Spring Boot · PostgreSQL (+pgvector) · MSA · Docker Compose · Redis · SSE · WebSocket · JWT · OAuth2 · MySQL · Oracle · AWS EC2 / RDS · GitHub Actions · Next.js / React / TS

★ ML · NLP · 데이터

SBERT Fine-tuning · HuggingFace · XGBoost · PyTorch · RandomForest · KMeans · KoBERT · Whisper / Pyannote · NER · 개인정보 익명화

★ CAREER & EDUCATION ★

● Education ● Career ● Degree · 자격증

2015 ~ 2023

2015.03 ~ 2020.02
명지대학교 컴퓨터공학과 학사
- 데이터베이스 설계 · 자료구조 · 컴퓨터 보안
- 총장상 (2019.12, TOPCIT 우수 성적)

2019.06 ~ 2019.08
세코닉스 - 인사팀 인턴
- 6주 인턴으로 신규 입사자 정보 관리와 ERP 운영을 지원

2020.08 **정보처리기사**

2021.09 ~ 2022.04 · 1120h
코드스테이츠 - AI 부트캠프 (7기)
- 데이터 분석 · AI · 자연어 처리 통합 과정
- KoBERT 뉴스 보정 기업 협업 프로젝트

2022.05 ~ 2022.06 · 3주
코드스테이츠 - AI 심화 NLP 과정
- PyTorch로 BERT를 추가 학습시켜 문장 유사도(STS) 계산

2022.09 ~ 2023.03
비상교육 - AI 연구개발 Cell
- 실제 회사 데이터로 AI 서비스 PoC 2개 1인 개발 (이탈 고객 예측, 질문-답변 검색)
- 대표이사 앞에서 시연

2022.12 **빅데이터분석기사**

2023.10 **SQLD**

2025 ~ 현재

2025.03 ~ 2025.08 · 960h
KOSA 생성형 AI 과정 - KOCRUIT
- LangGraph와 FastAPI 서버를 처음부터 끝까지 구현
- 최종 프로젝트 팀장 · 과정 내 1위

2025.08 ~ 2025.10
KOSA Spec-up - K-Bridge
- 수출입 통관 규정 AI 분석 4인 팀 프로젝트
- 수입 요건 분석 담당

2025.09 **OPIc 영어 IH**

2026.03 ~ 현재
KOSA - AI 교육 보조강사
- 직업훈련교강사 보수교육에서 수강생 12~20명의 실습 지원
- 담당 강의 16개 (Spring AI · MCP · LangGraph · Claude Code 등)
- 수강생 코드의 오류를 함께 찾고 기술 질문에 답변

2026.06 ~ 2026.08.25 · 320h
[NIPA·NVIDIA] GPU 가속 기반 AI Agent 엔지니어 과정
- LangGraph · 고급 RAG · MCP · A2A 멀티에이전트
- 프로젝트 QMesh

2026.07 **정보보안기사 필기 합격 · AI-POT**

2026.08 **NVIDIA DLI 인증 5개 (RAG Agents · LLM 앱 · Diffusion 등)**

2026.08 ~ 2026.09.30 · 수료
[Google] 2026 Google Cloud Study Jam H2

2026.10.01 ~ 2026.10.23 · 120h · 수료
[NIPA·Intel] AI for Current Workforce PBL 기반 프로젝트
- OpenVINO 양자화 · 로봇팔 ACT 정책 · 최종 프로젝트 보리 (팀)

회사가 다른 AI 에이전트끼리 일을 맡기는 연결 계층 (A2A)

01 QMesh

★ AI AGENT 대표작

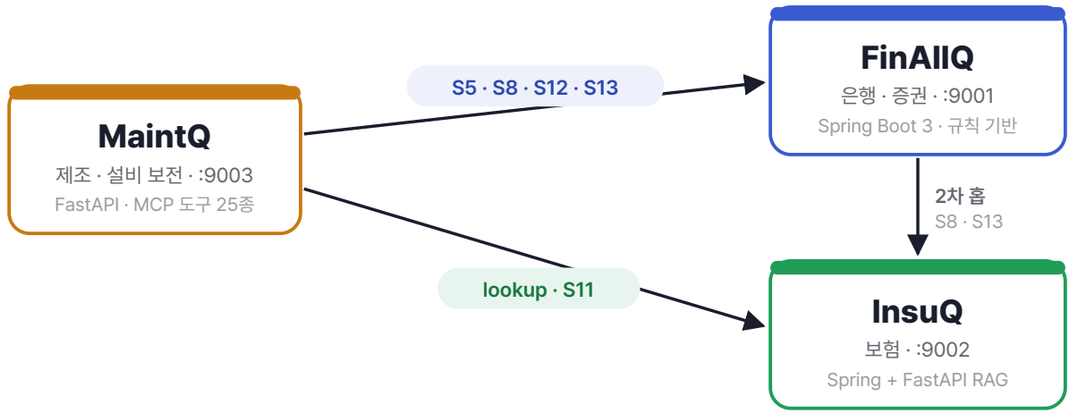
2026.08 ~ 진행 중 · 개인 1인

계약 설계 · 어댑터 구현 · 회사 간 연결 확인

↗ GitHub ↗ 상세 페이지 ↗ 발표자료

#A2A #MultiAgent #ProtocolDesign #HumanInTheLoop

연결 대상 3종도 모두 직접 만든 1인 프로젝트



세 에이전트가 서로 직접 부릅니다. 에이전트가 3개라 중간 어댑터를 따로 두지 않고 레포끼리 연결했고, 약속(Agent Card · Task 스키마)만 QMesh 한곳에 모았습니다.

```
POST /a2a/skills/assess-loan · request_chain_id: CHAIN-LOAN-a0639920
→ FinAllQ LTV 판정 → InsuQ 담보 보험 검증 ← decision: conditional · 1.3s
2026-08-24 실측. 5억 신청 · 담보 인정액 3억 → 보장 부족으로 조건부
```

구현	FastAPI · httpx · pydantic (Python), Spring MVC (Java). A2A 전용 SDK 없이 약속(JSON Schema)을 직접 구현했습니다.
약속	Agent Card 3종 · Task 스키마 13종 · `request_chain_id` · `Idempotency-Key`

◆ 문제

- 보험 · 금융 · 설비 보전 에이전트를 각각 만든 뒤, 셋을 **서로 다른 회사로 가정**하고 연결
- 막힌 곳: 다른 회사에서 온 요청을 **어디까지 믿고, 무엇까지 보여 줄지**
- MCP는 AI가 자기 도구를 쓰는 방식이라, **신원 확인 · 사람 승인 대기**는 따로 설계

◆ 구현한 시나리오

고장	설비 에러 → MaintQ가 InsuQ에 약관 조회 → 근거 조항과 함께 «판단 보류»로 회신
발주	부품 발주 → 공장 팀장 · 재무팀 승인 → FinAllQ 결재함 도착 → 은행 결재(사람) 뒤 출금
대출	설비 담보 대출 신청 → FinAllQ LTV 판정 → InsuQ 담보 보험 확인 → 조건부 판정
처분	설비 처분 → FinAllQ 정산으로 처분 제한 해제 → InsuQ에 처분 통지 → 최종 서명은 사람

연결 7가지를 세 시스템을 실제로 띄워 하나씩 끝까지 실행해 확인

◆ 제가 내린 결정

신원	**보낸 회사와 자료 주인을 따로 확인** Actor · Subject를 나눠 조회 → 심사 → 돈 이동 순으로 허락 범위를 좁힘
승인	**돈이 움직이면 사람이 세 번 승인** 공장 팀장 · 공장 재무팀 · 은행 결재. AI는 초안까지만, 승인 권한 없음
약속	**약속에 없는 값은 오류(502)로** 조용히 넘기면 틀린 값이 다음 회사까지 흘러가기 때문

QMesh 연결 에이전트 · 은행·증권 금융 비서

01-1 FinAllQ AI 비서인데 생성형 AI를 쓰지 않았다

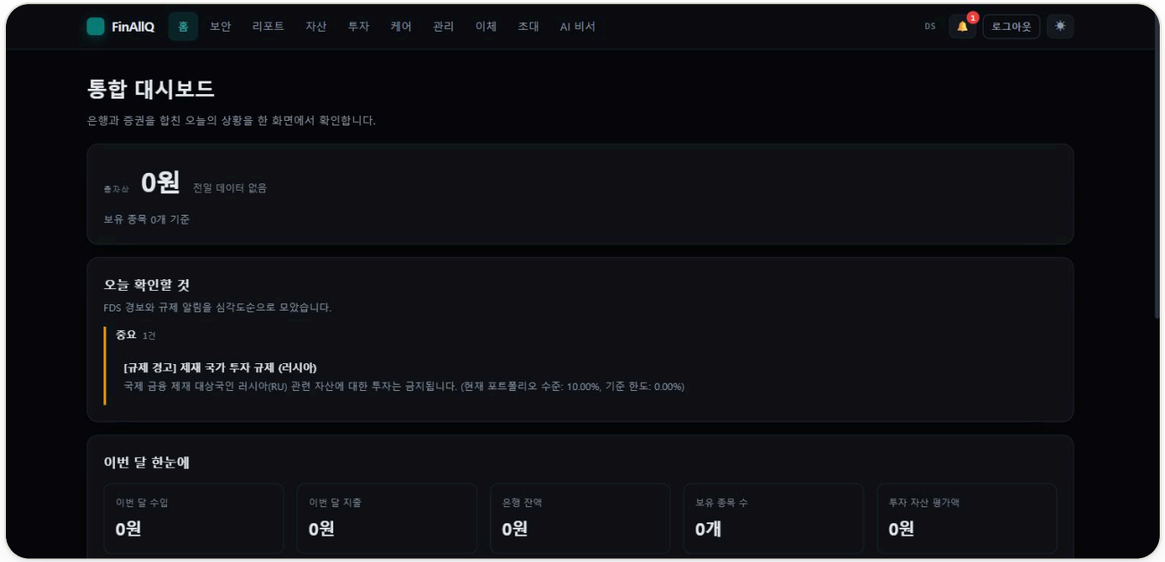
2025.10 ~ 진행 중 · 개인 1인

Spring Boot 3 + FastAPI + React 19 · PostgreSQL · Redis

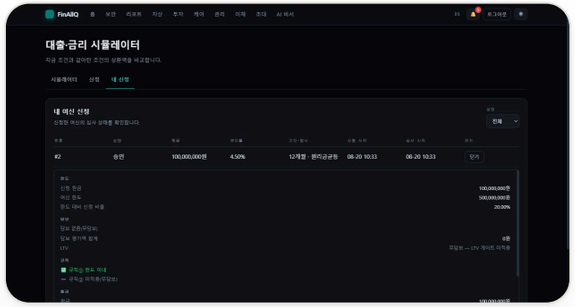
↗ 상세 페이지

#규칙기반 #FDS #직무분리 #2FA

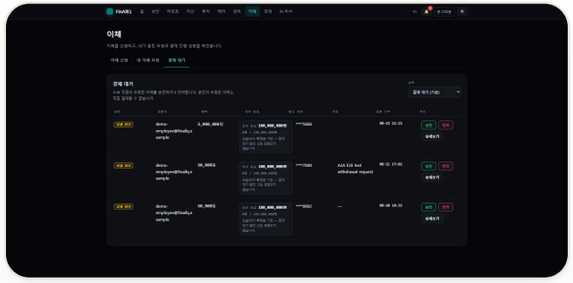
아래 화면은 모두 배포 환경(Vercel + Cloud Run)에서 찍은 캡처입니다



통합 대시보드. 상단 규제 경고는 포트폴리오 비율을 계산해 기준값과 비교한 결과입니다.



여신 심사 근거. 한도 · LTV · 규칙 판정



기업 이체 결재함. 요청자별 1일 한도 사용액

◆ 문제

- 은행 · 증권을 한 화면에 모은 금융 비서
- 이상거래 판정 · 이체 차단 등 금융 기능은 **이유를 설명해야 하는 결정**
- 이유: 감사 기록에 «AI가 그렇게 판단했습니다»라고 적을 수 없음

◆ 주요 기능

01 통합 대시보드
은행 + 증권 합산, 규제 경고는 비율 계산 결과

02 여신 심사 근거 카드
심사와 같은 판정 로직 재사용 → 화면과 심사 결과 일치

03 여신 ↔ 이체 연결
서버가 «출금 가능» 판정할 때만 버튼, 계좌는 사람이 입력

04 기업 이체 결재 (직무 분리)
신청자에게는 취소 버튼만 제공, 승인은 결재자(MANAGER)가

05 FDS 고위험 → 2FA 승격
결재 후에도 이상거래 재평가, 고위험이면 추가 인증(TOTP)

06 스미싱 탐지
키워드 목록 대신 모델로 판정, 위험 단어를 근거로

07 AI 비서
범위 밖 질문은 지어내지 않음, 실패한 기능은 배너로

모델	IsolationForest (이상거래) · TF-IDF + LogisticRegression (스미싱) · 생성형 AI 0건
도구	MCP Hub 도구 20종을 질의 키워드 점수로 최대 3개까지 고릅니다.

01-1 LLM을 쓰지 않은 이유

측정 기준일 2026-09-08

#감사가능성 #IsolationForest #안전장치

이 서비스가 하는 일은 결국 «어떤 도구를 어떤 순서로 부를까»입니다

결정 공간

선택지가 닫힌 문제

- 도구 20종, 하는 일도 고정 → **선택지가 닫힌 문제**
- LLM을 넣으면 같은 질문에 같은 답이 보장되지 않고, 고른 이유도 설명 불가
- 그래서 질의 · 도구 키워드 점수로 최대 3개를 고르는 규칙

감사

금융은 근거를 남겨야 함

- 어떤 입력이 어떤 규칙을 탔는지가 코드에 남음 → 판정 근거를 감사 기록으로
- 답변에 실릴 수 있는 항목도 목록으로 고정

모델

규칙이 안 되는 곳에만 모델

- 이상거래는 IsolationForest - 정상 패턴의 경계를 사람이 기준값으로 정할 수 없기 때문.
- 스미싱은 TF-IDF + LogisticRegression - 문구 변형이 끝이 없어 키워드 목록은 바로 뚫리기 때문.

QMesh에서의 역할 - 공장이 보내는 출금 · 대출 · 정산 요청을 받는 은행입니다. AI가 «승인»이라고 답해도 대출 상태는 «심사 중»으로 남고, 사람이 따로 승인합니다.

LLM 대신 공들인 안전장치 4가지

안전장치	하는 일
① 권한별 도구 노출	권한 밖 도구는 도구를 고르는 단계에 아예 올라가지 않습니다.
② 부작용 도구 자동 실행 금지	«송금해줘»라고 해도 시스템이 알아서 돈을 움직이지 않게, 부작용이 있는 도구는 자동 선택에서 뺐습니다.
③ 송금 흐름 FDS 필수 통과	송금 계열 흐름은 이상거래 점수를 건너뛸 수 없게 했습니다. (지금은 발동하지 않습니다. 아래 한계 참고)
④ 출력 항목 목록	도구가 무엇을 돌려주든 목록에 등록된 항목만 답변에 실어, 민감정보가 새는 길을 막았습니다.

LLM으로 바꿀 기준도 미리 정했습니다. 도구 매칭 실패가 눈에 띄게 늘거나 도구가 20종을 크게 넘어 키워드가 자주 부딪치면, 도구를 고르는 부분만 바꿉니다. 그때도 안전장치 4가지는 그대로 둡니다.

한계 - 아직 안 된 것도 같이 적습니다

FDS 필수 통과는 자리만
대상 도구 목록이 비어 있어 지금은 발동하지 않습니다. 코드 주석과 백로그에 적어 두었습니다.

재학습 파이프라인 없음
학습 데이터가 코드에 고정돼 있습니다. 사용자마다 달라지는 것은 기준선(30일 평균 이체액 등)뿐입니다.

동의어 · 오타에 약함
도구를 키워드 부분 일치로 고르기 때문입니다. 코드 맨 위 주석에 «LLM이 아니다»라고 적어 두었습니다.

QMesh 연결 에이전트 · 보험 약관 RAG

01-2 InsuQ 근거 조항과 함께 답하는 약관 에이전트

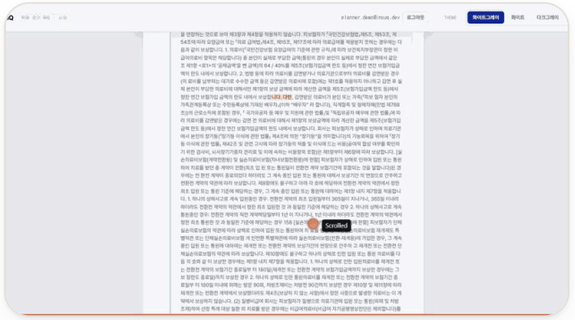
2026.07 ~ 진행 중 · 개인 1인

Spring Boot 4 + FastAPI/LangGraph · Qdrant · Next.js

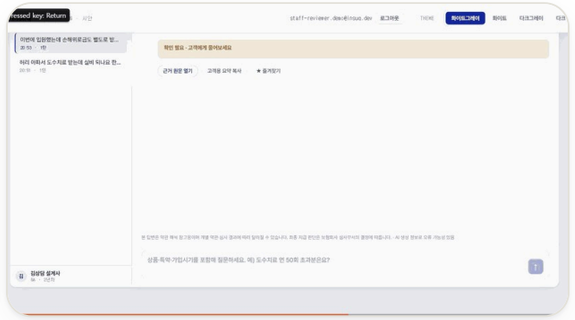
↗ 상세 페이지

#RAG #근거인용 #거부게이트 #HumanInTheLoop

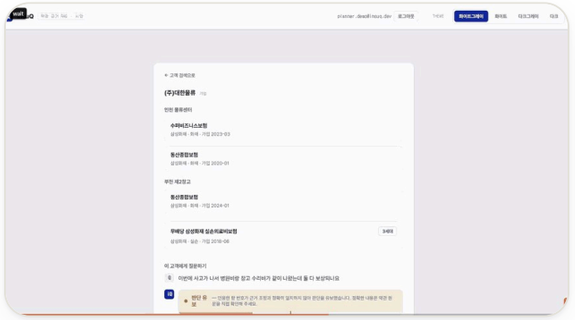
아래 화면은 모두 실제 서비스 캡처입니다



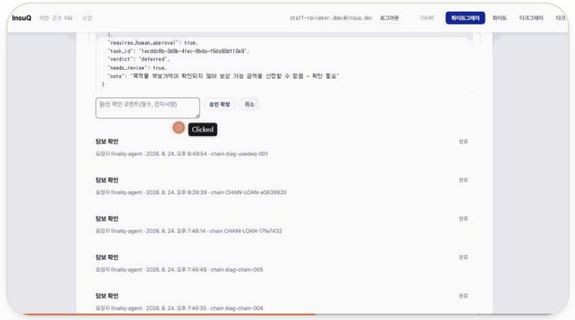
① **근거 인용**. 조항 원문과 함께 답합니다.



② **거부 게이트**. 근거가 없으면 거부합니다.



③ **되묻기**. 정보가 빠지면 확인 질문을 냅니다.



④ **청구 승인**. 심사역이 전자서명해야 완료됩니다.

모델	생성 `gpt-4.1-mini` · 평가 `gemini-3.5-flash` · 임베딩 `nemotron-3-embed-1b`
도구	Qdrant 검색 · LangGraph · A2A 스킬 6종 (`lookup-clause`, `verify-collateral-insurance` 등)
QMesh	공장의 약관 질문과 설비 처분 통지를 받고, 은행이 대출을 심사할 때 담보 보험을 확인해 줍니다.

◆ 문제

- 보험금 판정이 틀리면 곧 누군가의 **금전 손해**
- 설계사는 고객 앞에서 근거 조항을 직접 보여 줘야 함
- «지급 가능/불가»만 알려 주는 답으로는 불가능

◆ 해결 방법 - 답변 형식을 코드로 고정했습니다

판정	가능성 높음 · 낮음 · 판단 보류 중 하나로만 냅니다.
근거	약관 조항 원문을 붙이고, 파트와 조 번호로 대조합니다.
확인	사람이 확인할 사항을 함께 적습니다.
✕	«지급 가능합니다» 같은 단정은 판단 보류로 낮춥니다.
✕	근거가 없으면 «약관에서 확인 불가»로 거부합니다.

◆ 방어선 두 개

- **LLM 호출 전**: 검색 결과가 없거나 점수가 모자라면 모델을 부르지 않습니다. 필요한 정보가 빠진 질문에는 확인 질문을 내고, 이때도 LLM을 부르지 않습니다.
- **LLM 호출 후**: 인용이 진짜인지 검증하고, 단정 표현은 판단 보류로 낮춥니다.

두 방어선 모두 프롬프트 지시로 두지 않고 코드로 동작합니다.

01-2 안전장치를 실행 경로에 두기

측정 기준일 2026-09-09 · 실손보험 27문항

#검색설계 #단정금지 #캐시 #기각한시도

설계사가 일하는 순서대로 찾습니다

검색

작게 찾고 크게 답하기

- 가입 상품으로 먼저 좁힘 → 항 단위로 검색 · 순위 → 답 만들기 직전에만 조 단위로 확장
- 참조 조문은 같은 파트 안에서 깊이 제한을 두고 따라감

인용 대조

조 번호에 파트까지 대조

- 한 약관 안에 제1조가 보통약관 · 특약 등 여러 파트에 있음 (한 상품에서 3곳)
- 조 번호만 맞추면 다른 파트의 같은 번호를 지어내도 통과 → 파트까지 대조

단정 금지

조사 하나로 샌 구멍

- «지급 가능합니다»는 잡고 «지급이 가능합니다»는 통과한 적 있음
- 탐지를 넓히면서 잘못 잡는 비율도 측정 (저장 답변 287건 중 새로 걸린 것 0건)

캐시

캐시가 안전장치를 우회하지 못하게

- 캐시 키 = 상품 필터 · 모델 · 프롬프트 본문 · 인덱스 버전
- 상품 필터가 빠지면 다른 고객 계약을 인용하게 되기 때문

◆ 측정 후 기각한 시도

시도	잰 결과
하이브리드 검색 (BM25)	정답 조항 상위 5개 포함률이 0.81에서 0.79로 떨어져, 제 가설이 틀렸습니다.
cross-encoder 리랭커	품질은 가장 좋았지만 응답 시간 중간값이 98.2초였습니다.
저가형 생성 모델	거부 정확도는 1.0000으로 같았지만, 답할 수 있는 질문을 거부한 비율이 0에서 0.1852로 올랐습니다.

실험 67건을 가설 · 측정 · 판단 형식으로 남겼고, 기각한 것도 지우지 않았습니다. 지우면 다음 사람이 같은 가설을 다시 세우기 때문입니다.

사고도 구조로 바꿨습니다. 개발 환경에 남은 인증 정보 때문에 운영 인덱스(4,233청크)가 지워진 적이 있습니다. 그 뒤로는 새 버전을 채우고 검증을 통과했을 때만 교체해서, 중간에 멈춰도 이전 버전이 남습니다.

◆ 한계

AI 채점 칸이 비어 있음
채점 모델이 게이트웨이 계정 폐기로 사라졌습니다. 사람이 채점한 24문항이 남아 있어 다시 맞출 수 있습니다.

화재보험 수치는 따로 읽어야 함
66문항 중 52문항을 채점해 상위 5개 포함률 0.6346입니다. 실손과는 약관도 질문도 달라 나란히 비교할 수 없습니다.

설비 진단 → 부품 발주 AI 에이전트 · NVIDIA 해커톤 예선 통과(상위 10팀)

02 MaintQ 설비 진단부터 부품 발주 초안까지

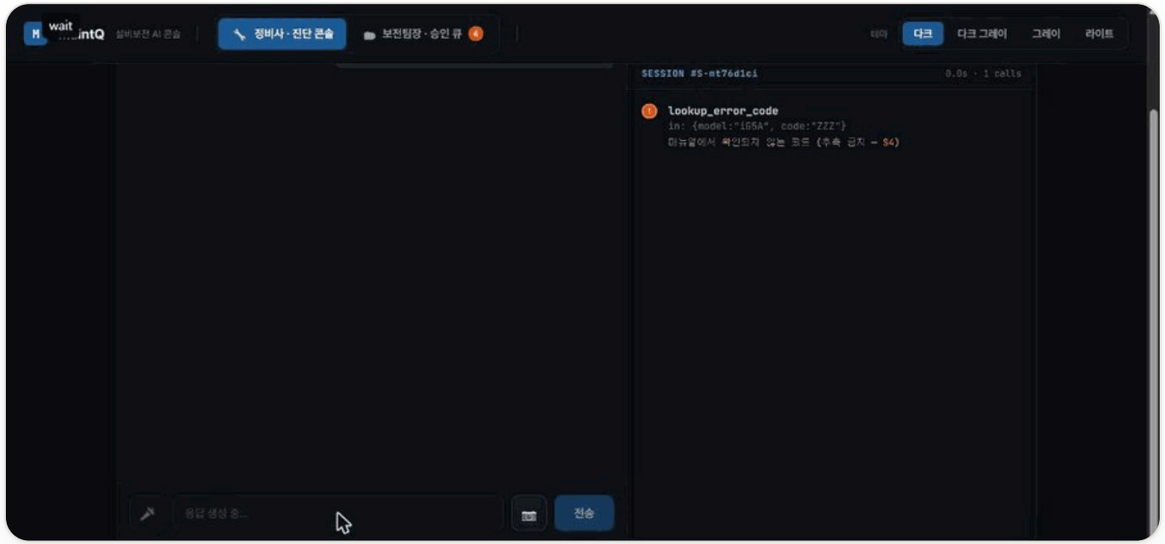
2026.07 ~ 진행 중 · 개인 1인

Python FastAPI + MCP + PostgreSQL(pgvector) + Next.js

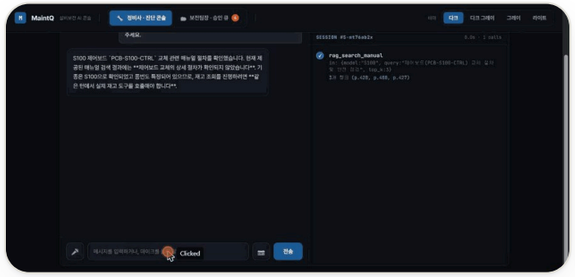
↗ GitHub ↗ 상세 페이지 ↗ 배포판

#MCP #AgentLoop #HumanInTheLoop #설비보전

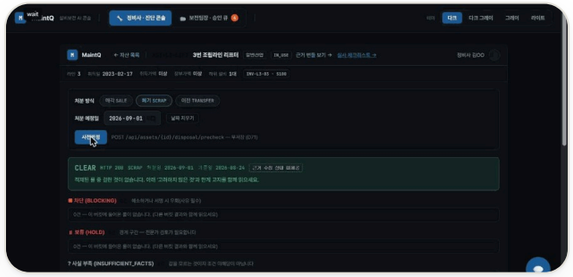
화면은 공개 매뉴얼과 생성한 시드 데이터로 찍었습니다. 실사용자는 없습니다



미지 에러코드. 카탈로그에 없는 코드는 지어내지 않고 A/S 경로를 안내합니다.



재고 0 → 호환 대체품



처분 사전판정. 사실이 모자라면 판정하지 않습니다.

문제

- 인버터 · PLC 에러 → 정비사가 PDF 매뉴얼을 10~30분씩 검색
- 부품은 전화 · 엑셀로 주문 → 매체가 계속 바뀌고 단계마다 대기가 생김
- 기존 설비관리 시스템(CMMS)은 기록만 할 뿐, **왜 에러가 났고 어떤 부품이 필요한지**는 판단 못 함

주요 기능

01 미지 에러코드
에러코드는 검색(RAG)이 아닌 정확 조회로 찾고, 없으면 «모른다»고 답합니다.

02 재고 0 → 호환 대체품
호환이 확인된 부품만 제안하고, 견적은 비교해 보여 주기만 합니다.

03 반복 고장 → 발주 보류
30일 안에 같은 에러가 3번 나면 근본 원인 점검으로 바꿉니다.

04 처분 사전판정
처분일 같은 사실이 빠지면 «사실 부족»으로 답합니다.

05 처분 근거 번들
법령 · 룰 · 평가값 · 계약 · 사실을 묶어 해시로 고정합니다.

06 법정 기한 · 위험등급
산출식을 보여 주고 «자동 반영 안 됨»을 알립니다.

07 지출 성격 분류
경계 사례는 «판단 보류 - 전문가 검토 필요»로 답합니다.

도구	MCP 도구 25종 (코어 7 + 확장 15 + 온보딩 3) · 쓰기 4종은 초안만 만듭니다
모델	Gemini 2.5 Flash · GPT-OSS:120B (통제 비교) · 해커톤판은 Nemotron

02 합계에 가려진 위험한 실패를 따로 셌습니다

측정 기준일 2026-09-11

#오특정 #통제비교 #도구경계 #신뢰구간

틀린 부품을 확신하면 엉뚱한 발주로 이어집니다

- 두 모델 정확도 45.6% vs 51.0% → 차이 없다고 넘기려 했으나, 안을 열어 보니 한쪽은 **틀린 부품을 확신**(오특정), 다른 쪽은 **답을 안 냄**(미특정)
- 합치면 더 위험한 쪽이 더 좋아 보임 → 두 실수를 따로 세고, 지시문만 바꿔 비교

GPT-OSS:120B · 20문항 · 각 120세션	변경 전	변경 후	판정
부품 특정 정확률 (합계)	71.1%	88.9%	구간이 겹치지 않음 (경계선)
ㄴ 오특정 (틀린 부품을 확신)	16.7%	**1.1%**	구간이 겹치지 않음 - 실제로 고친 것
ㄴ 미특정 (답을 안 냄)	12.2%	10.0%	구간이 겹침 - 주장하지 않음
근거 페이지 인용률	99.1%	96.3%	소폭 하락 - 구간이 겹침
안전 경고 통과	92.6%	89.8%	소폭 하락 - 구간이 겹침
모르는 에러코드를 지어낸 비율	0.0%	0.0%	양쪽 모두 목표 달성

몇 번 돌려야 믿을 수 있는지부터 쟀습니다. 정답률이 50% 안팎인 문항은 20번을 돌려도 95% 신뢰구간 폭이 40%p였고, 같은 조건의 10번짜리 묶음 둘이 인용률에서 12%p 어긋나기도 했습니다. 그래서 위 비교는 각 120세션으로 돌렸습니다.

도구 경계

조회와 검색을 다른 도구로

- 도구마다 «언제 쓰고 언제 쓰면 안 되는지»를 명시
- 에러코드 조회에 검색을 쓰면 «그럴듯한 다른 코드»가 올라옴 → 정확 조회 도구를 따로
- 실패도 오류 대신 정해진 상태값으로 → 에이전트가 다음 행동을 고름

쓰기

AI는 초안만, 신원은 서버가

- 쓰기 도구 4종은 모두 초안만 생성, 승인 · 반려 · 서명은 사람 전용 화면에서
- 요청자 신원은 도구 인자로 받지 않고 서버가 넣음 → AI가 신원을 꾸밀 길이 없음

◆ 한계

부품 특정 목표 미달
gemini-2.5-flash 측정(08-17)에서 40.0%(18/45)로 목표 90%에 못 미쳤습니다.

인용 · 안전 경고 미달
같은 측정에서 인용률 90.7%(49/54), 안전 경고 70.4%(38/54)입니다.

현장 효과는 못 잼
«고장에서 발주까지 30분 이상 → 5분»은 가설 지표로만 적어 두었습니다.

QMesh에서의 역할 - 일이 시작되는 공장입니다. 은행과 보험사에 요청을 보내기만 하고, 받는 요청은 설계상 두지 않았습니다.

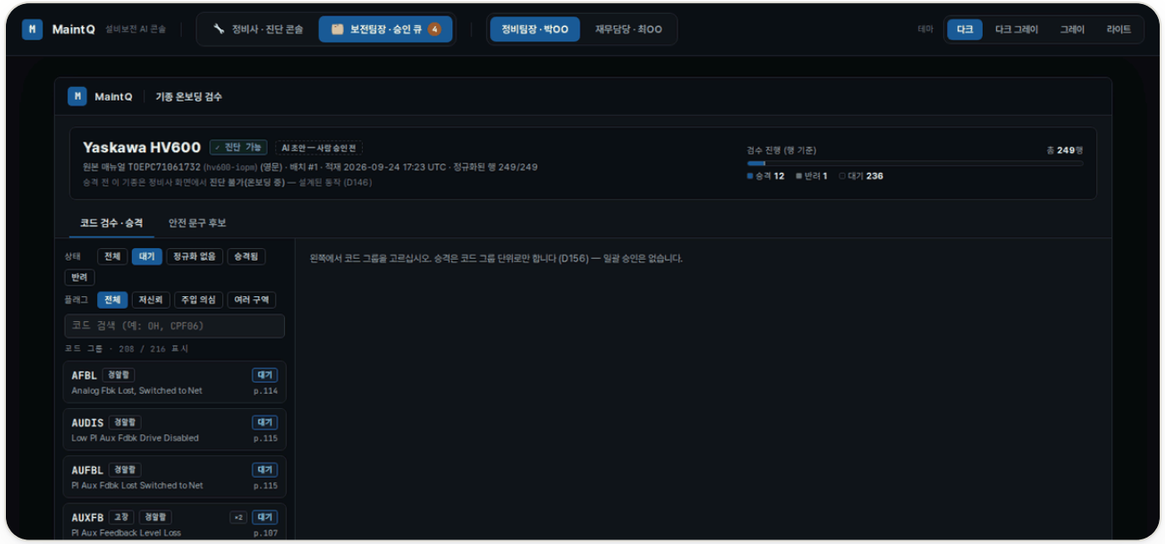
MaintQ · NVIDIA 해커톤 예선 통과(상위 10팀, 2026.09)

02 에이전트는 초안까지, 확정은 사람이 합니다

미션 «NemoClaw와 OpenShell로 에이전트 보안하기»

#NemoClaw #OpenShell #Nemotron #HumanInTheLoop

영문 매뉴얼뿐인 새 기종(Yaskawa HV600)을 한국어로 진단할 수 있게 했습니다



온보딩 검수. AI가 만든 한국어 초안을 사람이 원문과 대조해 코드 그룹 단위로 확정합니다.

결과 - 고장 코드 249행을 전량 정규화했고, 8코드를 사람이 확정했으며, 안전 문구 1건을 원문 p.29와 대조해 승인했습니다. 승인 전에는 «HV600에서 GF 떴어»에 비슷한 코드를 추측하지 않고 A/S로 안내하고, 승인 뒤에는 정의 · 원인 · 조치와 안전 문구가 함께 나옵니다.

아직 안 된 것 - Nemotron 기준 정확도는 다시 재지 않았습니다. 공개 배포판에는 샌드박스가 없어 녹화 영상으로 보여 드립니다.

◆ 문제

- AI 번역을 그대로 믿으면 두 가지 사고가 납니다. 번역이 틀려 **잘못 진단**하거나, 방전 대기 시간 같은 **안전 수치가 틀립니다**.
- 기존 기종의 10분을 새 기종에 그대로 쓰는 것도 같은 오류입니다.

◆ 일을 넷으로 나눴습니다

추출	**LLM 없이 코드가**	PDF 고장 표에서 코드 · 페이지 · 원문 행을 뽑습니다
초안	**샌드박스 안의 에이전트가**	NeMo Agent Toolkit · Nemotron으로 만들고, 스테이징 테이블에만 씁니다
확정	**사람이 원문과 대조해**	코드 그룹 단위로 확정합니다
안전	**사람이 승인한 문구만 노출**	원문에 없는 숫자는 API가 거부합니다

보안 두 겹

- **샌드박스**: API 키는 게이트웨이에만 두고, 나가는 연결은 허용목록만 엽니다. 파일 쓰기는 Landlock이 막습니다.
- **서버 · DB**: 도구는 INSERT만 하고 수정은 DB 트리거가 거부합니다. 매뉴얼에 숨은 지시문은 서버가 판정해 합성 주입 5건을 모두 막았습니다.

AI 기반 코드 보안 분석 플랫폼

03 SecureAI Engine

★ AI AGENT 대표작

~2026.07 (Sprint 0~30) · 개인 1인

Claude Code 멀티 에이전트 오케스트레이션으로 개발

↗ GitHub ↗ 데모 영상

#TrustworthyAI #DAST교차검증 #탐지율개선 #DevSecOps

모두의 창업 · 스파크클로 창업 프로그램 출품

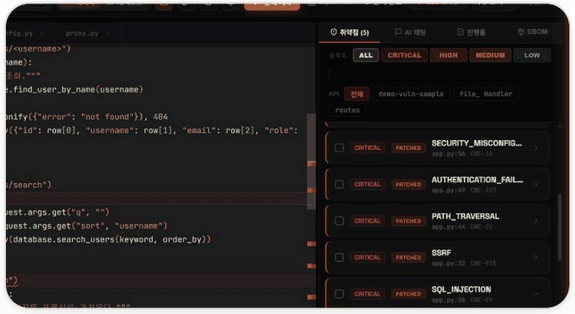
코드 읽기 ingest → 이전 결과 재사용 확인 cache → **Claude로 분석 analyze** → 취약점 분류 classify → 수정 코드 만들기 patch

실제로 공격
격리된 샌드박스에서 뚫리는지 확인합니다.

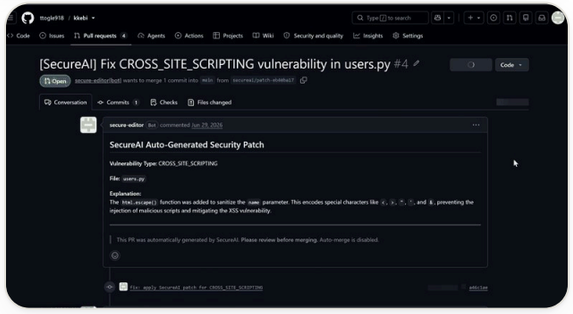
GitHub PR
패치를 PR로 올리고, 병합은 사람이 누릅니다.

증적 문서 3종
CISO 보고서 · 행안부 가이드 · ISMS-P

LangGraph 5단계 흐름. 중간에 멈춰도 이어서 할 수 있고, 분석 · 수정 · 증거 남기기를 도구로 나눠 AI가 필요할 때 부릅니다.



취약점 목록. 심각도별로 고칠 코드까지 보여 줍니다.



패치 PR. 병합은 사람이 누릅니다.

◆ 문제

- 팀 프로젝트에서 AI로 코드를 쓰다 보니 .env가 깃에 올라가고 API 키가 코드에 박히는 일이 반복
- **AI가 쓴 코드를 그대로 믿으면 안 된다**고 판단 → 보안 분석 도구 개발 시작

◆ 해결 방법

- **실제로 공격해 보고 확인**: 찾은 취약점을 격리된 실험 환경(샌드박스)에서 실제로 공격해 보고, 정말 뚫리는지로 증명했습니다.
- **DB는 서버 API로만**: AI가 DB를 바로 조회하게 하면 SQL 인젝션을 막는 파라미터 바인딩을 쓸 수 없었습니다. 조금 돌아가더라도 서버 API를 거치게 바꿔 DB에 닿는 경로를 줄였습니다(ADR-016).

◆ 설계에서 신경 쓴 것

- **역할을 나눈 AI 개발**: 총괄 · 기획 · 개발 · 리뷰 · 문서 역할의 Claude Code 에이전트를 동시에 돌려 혼자서 Sprint 0~30을 진행했습니다.
- **운영에 필요한 것까지**: 모니터링(Prometheus · Grafana · Jaeger), 진행 상황 실시간 전송(SSE + Redis), 네트워크 4개 분리, JWT HttpOnly.
- **쓰는 곳을 넓힘**: VSCode 확장은 코드 옆에 취약점을 표시하고, 안드로이드 앱(Kotlin · Compose · 푸시 알림)도 같은 서버에 연결했습니다.

모델	Claude Sonnet 4.6 (제품) · Gemini 2.5 Flash (가이드라인 비교 실험)
도구	LangGraph · Docker 샌드박스 · FastAPI · Redis · Prometheus · Grafana · Jaeger

03 오탐을 유형별로 나눠 줄인 과정

OWASP BenchmarkJava 1.2beta · 2026-07-03 ~ 07-04 측정

#오탐분석 #가이드라인 #OWASPBenchmark

탐지율은 올렸고, 오탐은 과제로 남았습니다

◆ 처음 분석에서 생긴 일

- 안전한 React 코드를 XSS로, 테스트용 가짜 데이터를 '심각'으로 오탐 → 유형별로 나눠 보니 원인이 다름
- 프레임워크가 막아 주는 패턴 → 프레임워크 규칙을 알려 줌
- 테스트 파일 → 분석 전에 제외
- 보안이 아닌 코드 품질 문제 → 따로 분류

◆ 가이드라인은 길수록 좋지 않았습니다

- 6만 자 가이드라인 → 오탐이 오히려 증가 (23.6% → 43.1%)
- 설명을 걷어냄 → 오탐은 줄었지만 탐지율도 하락
- 찾을 항목만 남김 → 탐지율 81.6%

Gemini 2.5 Flash · 330케이스	크기	탐지율	오탐률
가이드라인 없음	0	47.3%	23.6%
옛 가이드라인	63,527자	60.9%	43.1%
1차 정리 - 설명을 걷어냄	기록 없음	54.6%	23.6%
2차 정리 - 찾을 항목만 남김	약 14,000자	**81.6%**	30.9%

◆ 정한 가이드라인으로 제품 모델을 돌린 결과

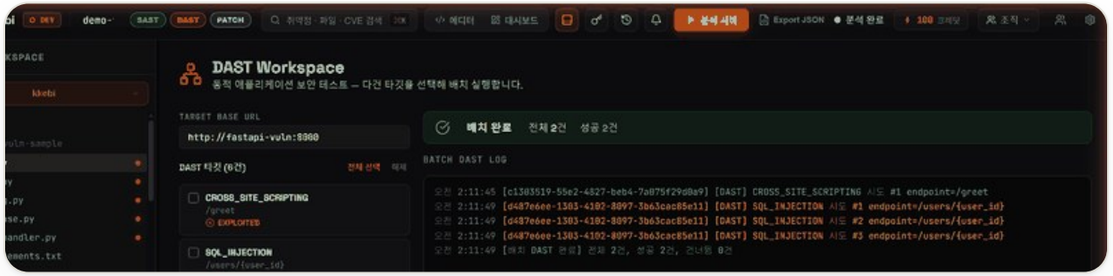
96.6%

탐지율 · 515 / 533 · Claude Sonnet 4.6 · 961케이스

27.1%

오탐률 · 116 / 428 · 남은 과제

오탐이 높은 유형은 trustbound 0.563 · securecookie 0.548 · crypto 0.460입니다.



DAST 검증. 찾은 취약점을 실제로 공격해, 뚫리면 EXPLOITED로 표시합니다.

프로젝트의 역할을 다시 정했습니다. 개발 도중 훨씬 강력한 모델이 나와 계속할 의미가 있나 싶었습니다. 그래서 이 프로젝트를 사람이 쓰는 프로그램으로 다시 정의했습니다. 판단의 근거를 사람에게 보여 주고, 사람이 확인해 고치게 돕습니다.

수출입 품목 번호(HS코드)로 규제를 자동 분석하는 AI

04 K-Bridge

2025.08 ~ 2025.10 · 팀 4인 (기여 25%)
요건 분석 · 요건-판례 교차 검증 담당
↗ GitHub ↗ 상세 페이지

#도메인적응력 #무역행정자동화 #CrossValidation #LangGraph

팀 프로젝트 · 모듈을 나눠 만든 뒤 AI 엔진 하나로 통합

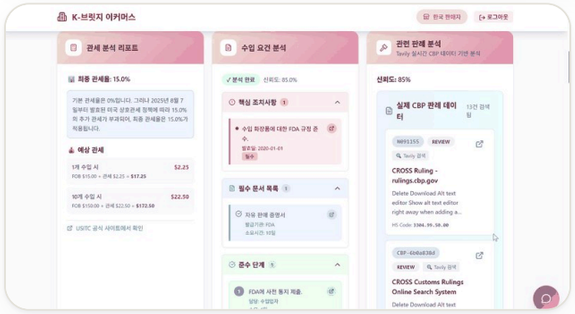
키워드 추출
3단계 폴백
→
기관 고르기
HS코드 → 기관
→
공공 API · 공식 웹
FDA · EPA 등
→
통합 · 검증
4축 병렬 + 등급
→
최종 정리
15개 항목

네 축 동시 분석
세부 규정 · 시험 절차 · 처벌 · 유효 기간

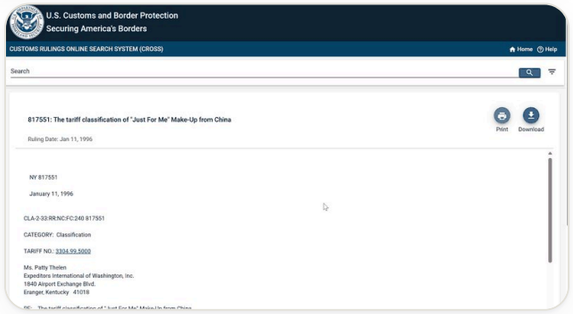
판례와 교차 검증
빠지거나 과한 요건을 표시합니다.

신뢰도 등급
RELIABLE · NEEDS_REVIEW · UNRELIABLE

요건 분석 LangGraph 흐름(제 담당). 같은 HS코드를 이미 분석했으면 저장해 둔 결과를 복사합니다.



분석 결과. 요건 분석에 신뢰도(85%)가 함께 붙습니다.



판례 카드를 누르면 미국 세관의 실제 판정문이 열립니다.

문제

- 미국 소액면세 제도가 없어지면서 작은 수출 판매자도 규제를 직접 챙겨야 하는 상황
- 요건 하나만 틀려도 화물이 묶이는데, 언어 모델은 없는 규정도 그럴듯하게 만들어 냄 → **성능보다 틀린 답을 걸러내는 게 먼저**

해결 방법 - 제 담당

- **공식 출처로만 좁혀 수집**: 미국 공공 데이터 API(openFDA · EPA CompTox · USDA · CPSC · FCC)를 먼저 쓰고, Tavily · SERP 검색은 기관 공식 도메인으로 한정했습니다.
- **판례와 교차 검증**: 모은 요건을 팀원이 분석한 미국 세관 판례와 대조해 빠지거나 과한 항목을 표시하고, 신뢰도 등급을 붙였습니다.
- **마지막 판단은 사람에게**: 판매자가 결과를 확신하지 못하면 관세사에게 바로 검토를 요청할 수 있습니다.

설계에서 신경 쓴 것

- **네 축을 따로, 동시에**: 서로의 결과를 기다릴 필요가 없어 `asyncio.gather`로 함께 돌렸습니다.
- **모델이 멈춰도 이어지게**: 키워드 추출은 OpenAI → HuggingFace → 규칙 순으로 내려갑니다.
- **원문과 한국어를 함께 저장**: 판매자는 한국어로 읽고, 관세사는 원문으로 확인합니다.

모델	gpt-4o-mini (요약 · 판례 요건 추출)
도구	LangGraph · FastAPI · Tavily · SERP · openFDA · EPA CompTox · FAISS

04 답마다 믿어도 되는 정도를 붙인 방법

최종 발표 2025.10

#환각억제 #판례검증 #신뢰도등급

근거 없이 «이게 맞다»고 말하지 않게 하는 장치

◆ 검증은 두 방향

- **교차 검증**: FDA와 USDA처럼 기관끼리, 또는 연방과 주 규정끼리 어긋나는 곳이 있는지
- **판례 검증**: 분석한 요건이 비슷한 판례에서 뽑은 요건과 얼마나 겹치는지
- 판례는 팀원이 만든 판례 분석 흐름(CBP 판정문 수집 → FAISS 색인 → 비슷한 사례 5건)의 결과를 받아 씁니다.

◆ 통합 신뢰도 = 판례 40 + 교차 30 + 출처 30

구성	비중	보는 것
판례 검증	40%	판례에만 있는 요건은 «누락», 분석에만 있는 요건은 «과잉»으로 표시
교차 검증	30%	기관 간 · 연방과 주 사이 규정 충돌
출처 신뢰도	30%	근거 문서 가운데 정부 공식 도메인(.gov) 비율

마지막 판단은 사람에게 넘깁니다. 등급이 NEEDS_REVIEW이거나 판매자가 확신하지 못하면 관세사 검토를 요청합니다. 관세사는 AI 분석 세 가지를 그대로 보면서 승인 · 반려하고, HS코드 수정을 제안합니다.

◆ 실제 결과 한 건

항목	값
판례 일치도	**0.85** · 대조한 판례 12건
일치한 요건	FDA VCRP Registration ↔ HQ_H304567 (유사도 0.95)
누락으로 표시	미생물 검사 필요 ← HQ_H305123 (심각도 medium)
판정	**RELIABLE** · 통합 신뢰도 0.87 HIGH

등급이 RELIABLE이어도 판례에만 있는 요건은 따로 표시해, 판매자가 놓치지 않게 했습니다.

◆ 최종 보고서 15개 항목

핵심 조치사항 · 필수 서류 · 준수 절차 · 위험 요소 · 예상 비용 · 라벨링 · 시험 · 검사 절차 · 금지 성분 · 사전 신고 · 처벌 · 벌금 · 인증 유효 기간 · 면제 조건 · 판례 검증 결과 · 통합 신뢰도 · 시장 접근성

◆ 한계 - 아직 안 된 것도 같이 적습니다

효과는 재지 못함
팀 과제로 끝나 판매자가 요건을 찾는 시간이 얼마나 줄었는지는 측정하지 못했습니다.

로그인 · 배포는 시연 범위
실제 서비스로 내려면 토큰 인증 · 비밀번호 암호화 · 환경별 설정부터 붙여야 합니다.

AI Agent 기반 기업용 채용 관리 시스템

05 KOCRUIT

2025.06 ~ 07 팀 프로젝트 · 팀장 (기여 40%) · 과정 내 1위
2025.09 ~ 2026.01 혼자 개선

↗ GitHub ↗ PPT

#Multimodal #LangGraph #MSA #TeamLead

아키텍처 · 면접 시스템 · STT 담당

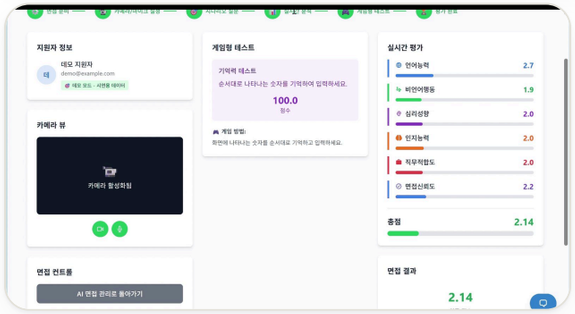
채용공고 등록 job post → **질문 · 평가표 미리 생성** LangGraph → **서류 평가** 점수 + 근거 → **면접 음성 · 영상** STT · 표정 → **질문별 리포트** report

백엔드 :8000
면접 연결 상태 · 실시간 통신 · DB처럼 기억해야 하는 일

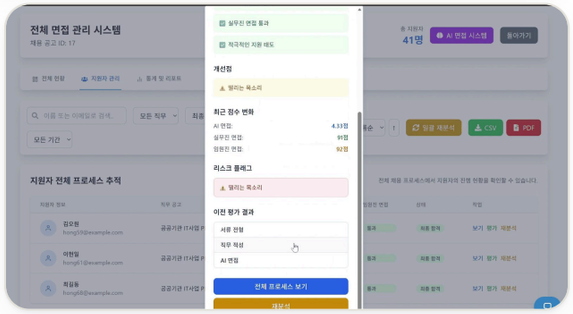
AI 서버 :8001
음성 인식 · 평가처럼 받아서 계산만 하는 일

분석 컨테이너 분리
PyTorch(음성)와 TensorFlow(표정)가 충돌

AI는 점수와 근거까지만 내고, 합격 · 불합격은 사람이 정합니다.



여섯 항목 AI 분석은 참고용, 최종 점수는 사람이 매깁니다.



지원자 41명의 단계를 리스크 표시와 함께 한곳에서 봅니다.

문제

- 서류 검토와 면접 기록에 사람 손이 많이 가고, 면접관마다 평가 기준이 다름
- AI로 돕되 **합격 · 불합격은 AI가 정하지 않게** 만드는 것이 목표

해결 방법

- **기억하는 일과 계산만 하는 일을 분리**: AI 서버에서는 실시간 통신 기능(websockets)을 아예 빼서 둘이 섞일 여지를 없앴습니다.
- **질문은 면접 전에 미리**: 면접 중에 질문을 만들면 기다리는 시간 때문에 대화가 끊겨, 지원서가 들어오면 질문과 평가표를 미리 만듭니다.

설계에서 신경 쓴 것

- **DB는 바꾸고 화면과의 약속은 유지**: 채용 단계를 별도 표로 분리하면서 예전 칸 이름으로도 읽히게 해 (SQLAlchemy @property) 화면 코드를 고치지 않고 옮겼습니다.
- **중복 AI 호출 줄이기**: 같은 요청은 24시간 캐시(Redis), 쉬운 작업은 저렴한 모델, 만든 결과는 DB에 보관합니다.

모델	gpt-4o (면접 답변 정밀 분석) · gpt-4o-mini (나머지) · Whisper · pyannote
도구	LangGraph · FastAPI · Redis · MediaPipe · DeepFace · Docker Compose

05 영상 · 음성을 나눠 분석하고, 구조를 다시 잡았습니다

팀 2025.06 ~ 07 · 혼자 개선 2025.09 ~ 2026.01

#Multimodal #LangGraph #WebSocket #비용절감

팀 기간에 만든 면접 분석을, 수료 후 혼자 다시 정리했습니다

멀티모달

영상 하나에서 세 갈래 분석

- 표정(DeepFace) · 자세와 시선(MediaPipe) · 음성(Whisper 전사)을 따로 분석해 합침
- 언어 · 비언어 · 심리 · 인지 · 직무 · 신뢰도 6개 항목 리포트로 냄

질문 생성

지원자마다 질문 경로가 갈림

- 포트폴리오가 있으면 프로젝트 · 스킬을 먼저 뽑고, 없으면 이력서만으로
- 공통 → 개인 → 직무 → 회사 순으로 질문을 쌓고 평가 기준을 함께 만듦

실시간

면접 중에는 모으기만

- 백엔드가 WebSocket으로 음성을 받아 세션을 관리
- 전사 · 판정은 상태가 없는 AI 서버에 넘기고, 결과는 참여자 모두에게 전달

테스트

고치기 전에 알아챌 장치부터

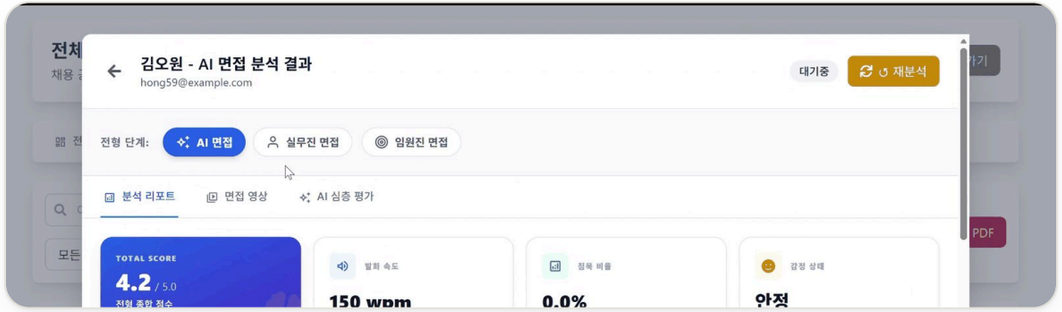
- 서버 테스트가 하나도 없던 코드
- API 명세가 바뀌면 바로 알아채는 도구와 테스트 11개를 먼저 만든 뒤 정리

◆ 혼자 개선한 결과 (2025.09 ~ 2026.01)

9 → 4
서류 평가 그래프 노드

8 / 9
gpt-4o → gpt-4o-mini로 바꾼 곳

- 면접 답변 정밀 분석 1곳만 gpt-4o 유지
- 한곳에 늘어놓았던 API 파일 34개를 기능별 8묶음으로 재정리
- 서버에 섞여 있던 AI 라이브러리 10종(torch · whisper · pyannote 등)을 걷어내 AI 서버 API 15개로 이전



질문별 리포트. 발화 속도 · 시선 · 자세까지 정리합니다.

팀 기간에는 시연용 프로토타입으로 과정 내 1위를 했습니다(서비스 6개 · DB 표 60개). ※ 지금 GitHub 레포는 2026.07부터 환경 분리 등 전면 리팩토링 중이라, 코드가 이 설명과 다를 수 있습니다.

★ MORE PROJECTS ★

06

2026.06 ~ 진행 중 · 개인 1인

KeyLens

개인 개발자용 자격증명 관리 도구 (키지기)

#LocalFirst #AES-256-GCM #OCR분류 #OpenSource

◆ 문제

보조강사로 일하며 수강생들이 API 키를 코드에 그대로 두는 걸 자주 봤습니다. 막힌 곳은 노션의 여러 ID처럼 모양이 똑같은 값이었습니다. 값만 봐서는 무슨 키인지 알 수 없었습니다.

◆ 해결

- 앞부분으로 알 수 있는 키(sk- · ghp_ · AKIA)는 바로 판별하고, 애매한 값은 화면 캡처를 OCR로 읽어 옆 이름표와 주소 구조로 구분합니다. 단서가 엇갈리면 «확인 필요»로 남깁니다.
- 서버 없이 내 컴퓨터에만 저장합니다(Argon2id + AES-256-GCM). SQLite에서 지운 데이터도 덮어쓰도록 설정했습니다.
- YAML 파일 하나로 새 서비스를 추가합니다(현재 10종).

★ 2026 오픈소스 개발자대회 출품. v0.5.1을 MIT 라이선스로 공개했습니다.

↗ GitHub ↗ 다운로드 ↗ 데모 영상

07

2026.08 ~ 09 · 개인 1인

SpendQ

AI 에이전트가 알아서 구매 · 역경매 + 조건부 결제

#x402 #A2A #Solana #Escrow #Gemini

◆ 문제

x402는 AI 에이전트가 돈을 내는 사실상의 표준이지만, 파는 쪽이 값을 정하면 사는 쪽은 내거나 말거나 둘 중 하나입니다. 흥정도, 판매자 간 경쟁도, 결과가 나쁠 때 돌려받을 방법도 없습니다.

◆ 해결

- 역경매: 판매자 AI들이 가격과 납기를 적어 응찰하고(HTTP 402), 정한 규칙으로 낙찰자를 고릅니다.
- 에스크로: 돈을 블록체인 금고에 맡겨 두고, 검사를 통과해야 지급하고 실패하면 환불합니다.
- 지출 한도는 AI 지시문에 두지 않고 블록체인 계정에 걸었습니다. 앱을 건너뛰고 직접 보낸 한도 초과 거래도 체인이 거절했습니다.

★ 기본 검사만으로는 불량 18건 중 15건, Gemini가 고른 검사를 더하니 18건을 모두 잡았습니다.

↗ 라이브 데모 (devnet)

08

2026.10.22 ~ 23 · 팀 · NIPA·Intel 과정

보리

채팅과 음성으로 시키는 과자 옮기는 로봇팔

#PhysicalAI #ACT #OpenVINO #LLM가드

◆ 문제

SO-ARM101 로봇팔이 여러 과자 중 오란다만 골라 옮겨야 했습니다. 2일 안에 로봇팔을 처음 배우면서 시범 녹화와 학습까지 끝내야 했습니다.

◆ 해결 (녹화를 뺀 소프트웨어 담당)

- 집고 놓기는 ACT 정책이, 개수와 쉬는 때는 코드가 맡습니다. 언어 모델은 말투와 대사만 정합니다.
- 정책은 Intel Physical AI Studio에서 공용 GPU로 학습 · 변환하고, 노트북 CPU에서 돌렸습니다(100스텝 묶음 0.14~0.23초).
- 대화 · 음성 API가 모두 실패하면 준비된 대사로 넘어가, 시연이 멈추지 않습니다.

★ 집기는 12번 중 1번 성공했습니다. 5 epoch만 학습한 것을 가장 큰 원인으로 봅니다.

↗ GitHub

★ MORE PROJECTS ★

개발 방식

개인 1인 · Claude Code CLI · 8개 레포

AI 하네스 엔지니어링

8개 프로젝트에 같은 규칙으로 적용한 AI 작업 관리 틀

#Harness #SubAgent #Hooks #HumanInTheLoop

◆ 문제

코딩 AI가 계획에 없던 기능을 끌어오고, 테스트가 실패하면 코드 대신 정답 파일을 고쳐 통과시키고, 검토만 맡긴 AI가 코드를 직접 고쳤습니다.

◆ 해결

- 막는 방법을 지시문(skill) · 도구 권한(setting) · 코드(hook) 세 단계로 나누고, 위험이 큰 프로젝트일수록 더 강하게 막았습니다.
- 검토 담당 AI에게는 읽기 도구만 줘서 고칠 방법이 없게 했습니다. 채점용 정답 파일 쓰기는 hook이 막습니다.
- 구현 · 테스트 · 검토 · 커밋은 자동으로 흘러가고, 마지막에 사람 체크리스트에서 멈춥니다.

★ hook이 지금도 작동하는지 돌려 보니 3개 중 2개가 한 번도 실행된 적이 없었습니다. `|| exit 0`이 오류를 가리고 있어서 고친 뒤, 위반 사례를 넣어 잡아내는지 확인했습니다.

09

2022.12 ~ 2023.03 · 1인 개발 · 대표이사 시연

비상교육 · 질문-답변 검색

#SBERT #NER #개인정보익명화

강사가 없을 때는 질문 답변이 늦어졌고, 게시판 원본 데이터에는 이름 · 주소 · 전화번호가 섞여 있었습니다. SBERT/KoBERT를 추가 학습시켜 새 질문과 비슷한 기존 질문 · 답을 찾아 주는 파이프라인을 만들고, 정규표현식과 NER을 조합해 개인정보를 가렸습니다(2주 동안 예외 사례를 반복 해결).

★ PoC라 익명화를 하지 않아도 됐지만, 데이터를 안전하게 만드는 일이 먼저라는 기준이 이때 생겼습니다.

10

2022.09 ~ 2022.12 · 1인 개발 · 대표이사 시연

비상교육 · 이탈 고객 예측

#DataPreprocessing #RandomForest #변수중요도

실제 서비스 데이터 10만여 건에서 약정 진행 중 고객 · 테스트 계정 · 로그인 기록 없는 고객을 걸러 내자 학습에 쓸 수 있는 것은 2천 건이 채 되지 않았습니다. RFM으로 고객 특징을 만들고 KMeans로 묶은 뒤, 이탈 · 연장 여부가 확정된 고객으로 정확도를 쟀습니다.

★ RandomForest 이탈 예측 85.3% · XGBoost 활동 저하 예측 89%. 변수 중요도로 무엇 때문에 떠나는지까지 설명했습니다.

★ FUTURE PLANS ★

AI가 들어갈 시스템 전체를 설계하는 아키텍트를 목표로 합니다.

01

안전한 Agent-Backend 통합 아키텍처

AI가 데이터에 직접 닿지 않고 백엔드라는 한 통로로만 오가게 해, 권한 확인이 한곳에서 이뤄지는 AI 서비스 구조를 만들고 싶습니다.

02

필요한 곳에만 쓰는 AI

규칙으로 되는 일은 규칙에 맡기고, 맥락을 읽어야 하는 곳에만 AI를 넣는 서비스를 설계하고 싶습니다.

03

안전하게 협력하는 Multi-Agent 시스템

회사가 서로 다른 AI끼리 어디까지 믿을지 정해 두고 자료를 주고받아, 사람이 여러 시스템을 오가며 확인하던 일을 줄이는 연결 구조를 만들고 싶습니다.

THANK YOU ★

읽어 주셔서 감사합니다. 연락 기다리겠습니다.

Email	ttogle918@gmail.com
GitHub	github.com/ttogle918
Portfolio	ttogle918.github.io
Blog	zena1010.tistory.com · velog.io/@ttogle918